← 返回记忆卡 打印 / 另存为 PDF ☑ 连「理解铺垫」一起印

点「打印 / 另存为 PDF」，在弹出窗口里把「目标」选为「另存为 PDF」即可保存。 每张卡的**结论**单独成块——想自测就把结论那半栏用纸盖住，看着标题先自己背一遍。 默认不印「理解铺垫」（那是理解用的长文，会让页数翻几倍）；需要就勾上面的选项。

操作系统 · 记忆卡

codebrick.tech · 用户ID 14127 · 2026-10-27 · 共 43 张

操作系统概述 8 张

中断与异常（OS 视角） 中频 对比表

中断 vs 异常（钉死）：

维度	中断（外中断）	异常（内中断）
来源	**CPU 外部**（I/O 完成、时钟）	**当前指令引发**（CPU 内部）
时序	**异步**	**同步**
检测时机	指令执行**完毕后**	指令执行**过程中**

- 中断 = 外中断、异步、来自 CPU 外部；异常 = 内中断、同步、由当前指令引发。

异常三类：

类型	性质	返回后	举例
故障 fault	可恢复	重新执行**当前**指令	缺页
自陷 trap	有意触发	执行**下一条**指令	系统调用
终止 abort	不可恢复	终止进程	硬件故障、非法指令

- 软中断 ≠ 硬中断：软中断由**指令**触发（属内中断范畴），硬中断由外部信号触发。

这是 os-intro 第 2 高频考点。最易混的是「中断 / 异常」这对术语：把它对成两组词——**中断 = 外中断 / 异步 / CPU 外部，异常 = 内中断 / 同步 / 当前指令引发**。来源决定了时序：外部信号何时来不可预测，所以异步；当前指令出错是执行到那一刻必然发生，所以同步。

怎么记不混（异常三类）：

- 按「**可不可恢复 + 主不主动**」三分：故障 fault 是被动出错但可救（缺页是典型，调入页后**重新执行当前指令**）；自陷 trap 是有意为之（系统调用，处理完执行**下一条**指令）；终止 abort 是不可恢复的硬错误，直接终止进程。
- 「返回后执行当前还是下一条」是高频陷阱：fault 重做当前指令、trap 走下一条，恰好相反，别记反。
- 软中断 ≠ 硬中断：软中断由指令触发、本质属内中断（异常）；这点常被题面拿来设套。

中断向量表、中断处理的硬件/软件分界（关中断→保存断点→送向量）属处理流程，OS 视角点到即止；要看完整时序点「展开博客精讲」或下方可视化。

系统调用 中频 易错辨析

系统调用 = 用户程序请求 OS 内核服务的唯一合法入口：

- 经 **trap / 陷入指令**主动进内核态执行；trap 本质是一种内中断（异常）。
- 凡涉及**资源管理**（I/O、内存分配、进程控制、文件读写）的操作，必须走系统调用。

系统调用 ≠ 库函数（钉死）：

维度	系统调用	库函数
执行环境	**内核态**	**用户态**（内部可能再调系统调用）
提供者	OS 内核	语言运行库
举例	read / write / fork	printf / malloc / strlen

- 库函数在用户态，**可能封装系统调用、也可能不**（如 strlen 纯计算不调）。
- 陷入指令（trap）本身**在用户态执行**，执行后才进内核态。
- 「涉及内核就是系统调用」是错的：异常（缺页、除零）也陷入内核，但**不是**系统调用。

这是 os-intro 第 1 高频考点（系统调用考过最多次）。核心一句：**系统调用是用户程序拿到 OS 服务的唯一合法方式**，因为用户态没有权限直接操作硬件资源，只能填「单子」让内核代办。判断题最爱问「以下操作一定 / 一定不触发系统调用」。

怎么记不混（三组易错对照）：

- **系统调用 vs 库函数**：库函数本身在用户态，它**可能**内部封装系统调用（printf 内部调 write、malloc 大对象调 brk/mmap），也可能完全不调（strlen、sin 纯用户态计算）。所以「库函数 = 用户态」对，但「库函数都不涉及内核」错。
- **陷入指令的位置**：trap 指令在用户态执行，是触发态切换的那一下；执行后才进内核态。别把「trap 在用户态」和「系统调用在内核态执行」搞混——前者说触发动作、后者说服务体。
- **不是所有进内核的都是系统调用**：缺页、除零是异常，也陷入内核，但不是系统调用；系统调用是用户**主动**请求服务，异常是**被动**出错。

  系统调用的执行流程（传参→trap→查系统调用表→执行→返回）属过程骨架，要看完整时序点「展开博客精讲」或下方可视化。

CPU 运行模式 低频 易错辨析

两种模式（靠 PSW 中的模式位区分）：

模式	别名	可执行指令
内核态	核心态、管态、特权态	所有指令（含特权指令）
用户态	目态	仅非特权指令

- **特权指令只能在内核态执行**：关中断/开中断、置时钟（装定时器）、I/O 指令、修改 PSW、修改页表/重定位寄存器。
- 非特权指令（用户态也能执行）：算术逻辑运算、访存、跳转。

态切换方向（钉死）：

- 用户态 → 内核态：**唯一入口 = 中断 / 异常（被动陷入）**。含外中断、异常、陷入 trap（系统调用）三类触发。
- 内核态 → 用户态：执行**一条修改 PSW 的特权指令**（如中断返回指令），把模式位改回用户态。
- 易混：系统调用的 trap 指令本身**在用户态可执行**，但执行后立刻进内核态。
- 态切换 ≠ 进程切换：态切换不一定换进程（系统调用后常返回同一进程）。

这是 os-intro 第 3 高频考点，「以下哪个是 / 不是特权指令」「以下哪个在用户态执行」是反复出现的题面。判断标准一句话：**凡是可能影响其他进程或整个系统的操作，都是特权指令**——所以关/开中断、I/O、修改 PSW、改地址翻译寄存器都是特权的。

怎么记不混（态切换两个方向）：

- 上行（用户态→内核态）记成「**被动陷入**」：用户程序自己没权限抬升模式位，只能靠中断/异常这类事件把它「拽」进内核态，这是**唯一入口**。三种触发：外中断（时钟/I/O）、异常（缺页/除零）、陷入 trap（系统调用主动发起）。
- 下行（内核态→用户态）记成「**主动放权**」：由 OS 执行一条修改 PSW 模式位的特权指令完成，所以下行必然在内核态发生。
- 两个易错点：① 系统调用用的 trap 指令本身在用户态执行，不是特权指令，但执行后立即进内核态；② 态切换比进程切换轻，态切换不一定伴随进程切换。

  命令解释程序（shell）属于用户态程序，缺页处理/进程调度/时钟中断处理在内核态——这类归属判断点「展开博客精讲」有速查表。

操作系统引导 低频 步骤口诀

引导骨架（逐级加载，次序固定）：

1. 通电，CPU 从固定地址取指 → 执行 **ROM 中的 BIOS/UEFI**（自检 POST，检测硬件）。
2. 读磁盘 **0 号块 = MBR**（主引导记录，含**磁盘分区表 + 引导程序**）。
3. MBR 中的引导程序扫描分区表，**找到活动分区**。
4. 读活动分区的 **PBR**（分区引导记录），加载启动管理器。
5. **加载 OS 内核**到内存（RAM）。
6. 内核初始化：建**中断向量表 / 页表 / PCB 等核心数据结构**，启动驱动，**创建初始/系统进程**。

- 链路口诀：**BIOS → MBR → PBR → 内核**。
- BIOS 固化在 **ROM**，开机最先执行；OS 内核最终被加载到 **RAM**。
- 为什么分这么多级：ROM 容量小只能放最小自举代码，逐级加载更大的引导程序。

引导是「程序加载所有程序，那第一个程序谁加载」的鸡生蛋问题，答案是逐级接力：固化在 ROM 里的最小自举代码（BIOS）跑第一棒，逐级把更大的引导程序拉进来，最终把内核装入内存。低频考点，记住链路骨架和两个落点即可。

怎么记不混：

- 链路口诀 **BIOS → MBR → PBR → 内核** 一气呵成：BIOS 找 MBR、MBR 找活动分区、PBR 加载内核。
- 两个高频落点：**BIOS 在 ROM**（开机最先执行、断电不丢）；**OS 内核最终加载到 RAM**（不是 ROM/BIOS——这是真题原题，os-2013-29 直接问「操作系统最终被加载到哪」答 RAM）。
- 内核初始化不是"只把代码搬到内存"：OS 还要建立中断向量表、页表、PCB/调度队列等运行所需的数据结构；其中**中断向量表**是 OS 接管中断/异常/系统调用入口的前提。
- MBR 在磁盘**第 0 号块（第 0 扇区）**，里面是「引导程序 + 磁盘分区表」两件套。
- 为什么要分级：ROM 太小装不下完整引导逻辑，只能放最小自举代码，逐级加载更大的程序。

  MBR 内部字节布局（446/64/2 字节）、BIOS vs UEFI 细节属背景，要展开点「展开博客精讲」或下方可视化。

操作系统概念与发展 低频 对比表

OS 四大特征（并发是最基本）：

特征	含义
并发	多个事件在同一**时间段**内发生（宏观并行、微观交替）
共享	资源可供多个并发进程共同使用
虚拟	把物理实体变为逻辑上的对应物（虚拟处理器/虚拟内存）
异步	进程以不可预知的速度推进

- **并发 + 共享是最基本的两个特征，互为存在条件**；虚拟、异步以二者为前提。
- 并发 ≠ 并行：并发是同一**时间段**内交替（单核也能并发）；并行是同一**时刻**同时（需多核）。

发展阶段对比：

阶段	关键词	目标
多道批处理	多作业驻留内存交替、无交互	**吞吐量**
分时	时间片轮转、多终端交互	**交互性 + 响应**
实时	截止时间约束	**及时 + 可靠**

- 多道批处理「无交互、追求吞吐」vs 分时「有交互、追求响应」。
- 实时分**硬实时**（必须在截止前完成，如导弹控制）/ **软实时**（偶尔超时可接受，如视频播放）。

四大特征是 OS 的宏观框架，选择题最爱在「哪个是最基本特征」和「并发 vs 并行」上设陷阱。把两句钉死：**并发与共享互为存在条件、是最基本的；单核能并发但不能并行**。常见错误说法「单核 CPU 不能实现并发」是错的——单核靠时间片轮转就能并发（宏观并行、微观串行），单核做不到的是并行。

怎么记不混（发展阶段的目标）：

- 三类系统各自的「目标 / 响应」是陷阱高发区。把目标记死：批处理追**吞吐量**（不是短响应时间）、分时追**交互 + 响应**（不是高吞吐）、实时看**最坏响应时间**（不是平均响应时间，必须 < 截止时间）。
- 「多道批处理 = 多核并行」是错的：多道批处理在单核也能实现，靠 I/O 与 CPU 重叠提升利用率。
- 分时系统的时间片不是越短越好：太短则切换开销大，要折中。

  各阶段的具体年代、代表机型属于背景，不进卡；要展开看「展开博客精讲」。

程序链接与装入 冷门 对比表

三种链接方式：

方式	时机	特点
静态链接	运行前	装入前合成一个完整装入模块
装入时动态链接	装入内存时	边装边链，便于模块更新
运行时动态链接	运行时	用到才链（.dll/.so），**最省空间、最灵活**

三种装入方式：

方式	地址转换时机	装入后可移动	硬件支持	适用
绝对装入	编译时定地址	否	无	单道
可重定位装入（静态重定位）	装入时一次改完	**否**	无	早期多道
动态运行时装入（动态重定位）	运行时转换	**是**	重定位（基址）寄存器	现代 OS

- 静态重定位：装入时把逻辑地址一次性改成物理地址，改完地址写死，**不能再移动**。
- 动态重定位：运行时靠**重定位寄存器**做 `物理地址 = 基址 + 逻辑地址`，程序**可在内存中移动**，支持虚拟内存。

链接管「多个模块怎么合成可执行文件」，装入管「可执行文件怎么进内存定地址」，是两件事别混。选择题最爱在「静态重定位 vs 动态重定位」上设陷阱——两者都在多道环境用，区别全在「装入后能不能移动」。

怎么记不混：

- 链接三类按「**链得越晚越省空间**」排序：静态链接运行前就全打包（文件最大、库代码可能重复）；运行时动态链接用到才链，多进程共享同一份库，**最省内存、最灵活**。
- 装入三类按「**定地址越晚越灵活**」排序：绝对装入编译时就钉死物理地址（只能单道）；静态重定位装入时一次性改完、改完不能动；动态重定位运行时才靠重定位寄存器换算，**唯一能在内存里移动**的，也是现代 OS + 虚拟内存的基础。
- 一句钉死：**只有动态重定位装入后可移动，且唯一需要重定位（基址）寄存器**。

  编译→链接→装入的完整流水线（.c→.o→可执行→内存）属背景，地址换算属计算（交给题）；要展开点「展开博客精讲」。

OS 结构 冷门 对比表

宏内核 vs 微内核（核心对比）：

维度	宏内核（大内核）	微内核
内核态放什么	OS 主要功能全在内核态	只留最基本功能（进程管理/调度、低级内存、IPC、中断）
性能	**高**（模块间直接函数调用）	低（频繁态切换、消息传递开销）
稳定性/可靠性	低（一损俱损）	**高**（一个服务崩不影响内核）
扩展性	差	**好**
代表	Linux、Windows	Mach、QNX、MINIX

- 微内核把文件系统、设备驱动等移到**用户态服务进程**，靠**消息传递**通信，故频繁态切换、性能差。
- 一次请求文件服务：宏内核 **2 次**态切换，微内核至少 **4 次**——这是微内核慢的根因。

其他结构：

- 分层：按功能分层，每层**只能调用紧邻下层**；便于调试但层间通信开销大。
- 模块化：划分独立模块、定义接口，模块间**可相互调用**（不限于相邻层）。
- 外核：**不做抽象，直接把物理资源分配给应用**，应用自行管理。

OS 结构的考点高度集中在「宏内核 vs 微内核」的对比。一句话抓住本质：**微内核把功能搬出内核态以换可靠性和扩展性，代价是频繁态切换导致性能差**。宏内核反之——全塞内核态，性能高但庞大难维护、一个模块崩可能拖垮整个内核。

怎么记不混：

- 用「态切换次数」记性能差异：宏内核一次文件请求是 用户→内核→用户（2 次）；微内核要 用户→内核→文件服务→内核→用户（≥4 次），切换越多越慢。
- 微内核留在内核态的是**最基本**的几样：进程管理/调度、低级内存管理、IPC（进程通信）、中断处理；文件系统和设备驱动都被赶到用户态当服务进程。
- 别把「分层」和「模块化」搞混：分层**只能调相邻下层**（约束强），模块化模块间**可互相调用**（更灵活）。外核的关键词是「**不抽象、直接给物理资源**」。

  各结构的代表系统、历史演进属背景，要展开点「展开博客精讲」。

虚拟机 冷门 对比表

两类 VMM（虚拟机监控程序 / Hypervisor）：

维度	第一类（Type 1，裸机型）	第二类（Type 2，寄居型）
VMM 运行在	**硬件**之上	**宿主 OS（Host OS）**之上
性能	**高**（无宿主 OS 开销）	较低（多一层 OS）
典型产品	VMware ESXi、Xen	VMware Workstation、VirtualBox
场景	数据中心、云计算	个人开发、测试

- VMM 作用 = 虚拟化硬件、让多个 OS 共享一台物理机并相互隔离。
- Guest OS 在虚拟机中运行在**用户态**，无法直接执行特权指令；由 VMM 用「陷入-模拟」等方式代为处理。

虚拟机是 os-intro 最冷门的考点（rare），了解两类 VMM 的区别即可。一句话抓本质：**区别全在 VMM 骑在谁上面**——第一类直接骑硬件（裸机型，快），第二类骑宿主 OS（寄居型，多一层故慢）。

怎么记不混：

- 「裸机 vs 寄居」对应「快 vs 慢」：Type 1 直接管硬件没有宿主 OS 抽税，性能高、用于云/数据中心；Type 2 是装在 Windows/macOS 上的一个软件（如 VMware Workstation），方便但慢一层，用于个人测试。
- 核心机制点：Guest OS 被降到**用户态**跑，它执行特权指令时会陷入，VMM 捕获并模拟（陷入-模拟）——所以「VMM 与 OS 特权级相同」是错的（真题 os-2025-24 的错误项）。
- VMM 的职责是**虚拟化硬件 + 隔离**，让一台物理机变多台、各跑独立 OS。

  二进制翻译、半虚拟化、硬件辅助（VT-x/AMD-V）等虚拟化实现细节属深水区，408 只考两类区别；要展开点「展开博客精讲」。

进程与线程 12 张

CPU 调度算法 高频 对比表

各调度算法特点对比（**只背特点，不背甘特图算周转/等待**）：

算法	是否抢占	是否饥饿	需预知服务时间	一句话特点
FCFS	**非抢占**	不会	否	简单公平，护航效应（短作业被长作业挡）
SJF	**非抢占**	会	**是**	非抢占中平均等待最短
SRTN（最短剩余时间）	**抢占**	会	**是**	所有算法中平均周转最优
RR（时间片轮转）	**抢占**	不会	否	时间片太大退化 FCFS、太小开销大
优先级	**可抢占 / 不可抢占两版**	会（静态）	否	可静态/动态优先级，有优先级反转
HRRN（高响应比）	**非抢占**	不会	是	兼顾长短作业，等越久响应比越高
MLFQ（多级反馈队列）	**抢占**	会	否	综合折中，新进程入最高级、用完降级

- **不会饥饿**：FCFS / RR / HRRN。**会饥饿**：SJF / SRTN / 静态优先级 / MLFQ。
- 响应比 R =（等待+服务）/服务 = 1+等待/服务（公式名记住，数值算交给题）。

这张卡只干一件事：**横向对比各算法的定性特点**，帮你在选择题里秒判「哪个会饥饿/哪个抢占/哪个要预知时间」。具体甘特图怎么画、周转等待算多少，**交给题和模拟器，本卡一律不背**。

先把「是否抢占」这一列钉死（最高频考点）：

- **非抢占**：FCFS、SJF、HRRN——这三个都是「选中了就让它跑完」。
- **抢占**：SRTN、RR、MLFQ——新进程到达/时钟中断会夺 CPU。
- **优先级调度有两版**：可抢占（高优先级新进程立刻夺）和不可抢占（等当前进程让出再比），做题看题目说哪种。
- 一个易混点：**SJF 非抢占、它的抢占版叫 SRTN（最短剩余时间优先）**——别把 SJF 当抢占的。

「饥饿」这一列怎么记：凡是「偏心某类进程」的就会饿死另一类——SJF/SRTN 偏心短作业（长作业被源源不断的短作业挤掉）、静态优先级偏心高优先级、MLFQ 长作业降到底层后被新进程不断抢占。反之 FCFS（按到达顺序，人人有份）、RR（轮流坐庄）、HRRN（等越久响应比越高、迟早轮到）**不会饥饿**。

几个标志性结论：FCFS 有「护航效应」（一辆大卡车堵住后面小轿车）；RR 时间片**太大退化成 FCFS、太小切换开销大**；MLFQ 规则=新进程进最高优先级队列、时间片用完降一级、高优先级队列非空才轮到低级。

> 响应比、各算法甘特图演练用配套模拟器；HRRN/优先级的逐步计算交给题。

CPU 调度基本概念 高频 易错辨析

调度三层次 + 「能不能调度」辨析：

层次	别名	频率	干什么
高级	作业调度	最低	外存后备作业→内存
中级	内存调度 / 对换调度	中	挂起↔激活（换出/换入）
低级	进程调度	**最高**	就绪队列→CPU

- **抢占式 vs 非抢占式**：非抢占只在进程**主动让出**（结束/阻塞）时才调度；抢占式新进程到达可夺 CPU。
- **不能调度的时机 = 关中断期间 / 原语执行中 / 中断处理过程中**。
- ⚠️ **临界区内仍可被调度**——临界区只保证对临界资源的**互斥访问**，不阻止 CPU 调度切换。「临界区内不能切」是错误表述（特例：修改就绪队列等内核数据的**内核临界区**才不能调度，普通临界区可以）。
- 闲逛进程（idle）：无就绪进程时运行，**优先级最低**。

这张卡的最高优先级是把一条高频错误掰过来：**临界区内到底能不能被调度**。

P0 辨析——临界区可以被调度：

- 临界区的职责只有一个：保证**对临界资源的互斥访问**（同一时刻只一个进程进临界区）。它**不阻止 CPU 切走**——进程在临界区里照样可能因时间片到、被抢占而让出 CPU，等会儿再切回来继续。互斥锁还在手上，别的进程也进不来，数据一致性不破坏。
- 所以「进程在临界区中不能进行处理机调度」是**错的**。真正**不能调度**的只有三个时机：**关中断期间**（连中断都响应不了，更别说调度）、**原语执行中**（原语是不可分割的原子操作）、**中断处理过程中**。把这三个记成「不能切的三件套」。

调度层次记法：高级管「作业进不进内存」（最少发生）、中级管「挂起进程换入换出」、低级管「就绪进程上不上 CPU」（最频繁）。频率从低到高，颗粒度从粗到细。

抢占 vs 非抢占：非抢占式只在进程**自己交出 CPU**（运行结束或主动阻塞）时才重新调度；抢占式允许更高优先级的新进程**夺走**当前 CPU，响应更快、适合交互系统。

> 周转时间/带权周转/等待时间这些**计算**交给题和模拟器，本卡只锚概念；优先级反转、动静态优先级等见博客。

死锁 高频 对比表

死锁四必要条件（缺一不可，同时满足才**可能**死锁）：
互斥 / 占有并等待（请求并保持）/ 不可剥夺 / 循环等待。
⚠️ 四条件是**必要非充分**：满足四条件不一定死锁；但死锁一定同时满足这四条。

预防四手段 ↔ 破坏的条件（一一映射）：

预防手段	破坏的必要条件
把互斥资源改造成共享（如 SPOOLing）	破坏**互斥**
运行前一次性申请全部资源	破坏**占有并等待**
申请不到就释放已持有的资源	破坏**不可剥夺**
资源编号、按编号递增顺序申请	破坏**循环等待**

处理三策略对比：

策略	思路	资源利用率	限制
预防	破坏四条件之一	低	限制最强
避免	分配前判**安全状态**（银行家算法）	中	须预知最大需求
检测与解除	允许发生，再化简资源分配图救	高	事后补救

（另有「鸵鸟策略」=不管，利用率最高。）

死锁 vs 饥饿 vs 活锁：

现象	进程状态	本质
死锁	**阻塞**，互相等待循环成环	涉及一组进程，永不解开
饥饿	就绪/阻塞，长期得不到资源	调度/分配长期不公（如读优先饿死写者）
活锁	**未阻塞**，一直在「让来让去」空忙	状态不停变化但无实质推进

死锁是「一组进程互相等对方手里的资源，谁也走不了」的僵局。最该背牢的是**四必要条件是必要非充分**：选择题最爱的干扰项就是「满足四个条件就会死锁」——错，它们是必要条件，循环等待存在但资源充足时也未必死锁。

预防四手段和四条件的一一映射是这张卡的核心：每个预防手段都精确对应破坏掉一个条件。配合哲学家进餐记更牢——限人数/奇偶拿筷破坏「循环等待」，同时拿两只破坏「请求并保持（占有并等待）」。

三策略的排序逻辑是「限制越松、利用率越高」：预防一刀切（最严、利用率最低）→ 避免靠银行家算法运行时试算（须预知最大需求）→ 检测与解除最宽松（先让它发生再救）。注意安全状态、不安全状态、死锁的包含关系：安全一定不死锁，死锁一定不安全，但不安全只是**可能**死锁。

死锁/饥饿/活锁三者别混：死锁的进程是**阻塞**且成环的一组；饥饿是被长期「不公平」地晾着（读者写者里写者被饿死就是典型）；活锁的进程**没阻塞**，还在不停动作（互相礼让 / 重试）却始终没有实质进展。

银行家算法的 Max/Allocation/Need/Available 矩阵数值演算、安全序列逐步求解、资源分配图化简过程属于推导，交给题与可视化。卡里只锚「四条件、四手段映射、三策略对比、三态辨析」这层结论。

进程与线程基本概念 高频 对比表

进程 vs 线程「基本单位」对比：

维度	进程	线程
是什么的基本单位	**资源分配**	**CPU 调度（处理机分配）**
地址空间	独立	同进程内共享
存在标志	**PCB（唯一标志）**	TCB
切换开销	大（切地址空间）	小

- **进程 = 资源分配基本单位；线程 = CPU 调度基本单位**（引入线程后调度单位从进程变线程）。
- 同进程内的线程**共享**：地址空间 / 代码段 / 数据段 / 全局变量 / 打开的文件 / 堆。
- 每个线程**独占**：线程 ID / 程序计数器 PC / 寄存器组 / 栈。
- **PCB 是进程存在的唯一标志**——OS 靠它感知和管理进程；进程 = PCB + 程序段 + 数据段，PCB ≠ 进程本身。

「资源分配 vs 调度」这组对比是本 KP 的命门，几乎每道相关真题都在这条线上设陷阱。把它钉死：**进程管资源、线程管上 CPU**。引入线程之前进程一肩挑两职（既分资源又被调度），引入线程后职责拆开——进程只留资源分配，调度交给线程。

怎么记「谁共享、谁独占」不混：

- 反过来想线程为什么轻量——它**共享**了进程几乎所有资源（地址空间、全局变量、打开的文件），所以创建/切换都省了「分配资源」这步，自然快。
- 但每个线程要能独立跑一条控制流，就**必须各自有**一套「执行现场」：PC（执行到哪了）、寄存器（算到一半的中间值）、栈（函数调用链和局部变量）。一句话——**共享的是「家当」，独占的是「执行现场」**。

PCB 的两个易错点：

- 「PCB 是进程存在的唯一标志」≠「PCB 就是进程」。撤销进程时撤的是 PCB+程序段+数据段，磁盘上的程序文件还在。
- 一个程序可对应多个进程（开两个记事本），别把程序和进程混为一谈。

进程的四特征（动态/并发/独立/异步，动态性最基本）、PCB 信息分类等铺垫可点「展开博客精讲」。

信号量 高频 步骤口诀

信号量 S 是整数，只能经原子操作 P（wait）/ V（signal）访问。

P(S)：S--；若 S<0 则把当前进程加入等待队列并自我阻塞 block()。
V(S)：S++；若 S≤0 则从等待队列唤醒一个进程 wakeup()。

S 值的物理含义（**仅记录型信号量成立**）：

- S>0：还有 S 个资源可用
- S=0：资源刚好用完，无进程等待
- S<0：**|S| = 正在等待该资源的阻塞进程数**

> ⚠️ 上面「S<0 时 |S|=阻塞进程数」只对**记录型信号量**成立——它有等待队列、会 block 让权。**整型信号量**是忙等（do-while 空转），值不会被减成负数，没有「负值=等待进程数」这回事。

两种用法（口诀）：

- **互斥**：mutex 初值 **1**；P、V 在**同一进程内成对**夹住临界区。
- **同步**：初值 **0 或 n**；P、V 分布在**不同进程**里，「前 V 后 P」实现次序。

P/V 必须成对：漏 P → 互斥失效；漏 V → 死锁。

信号量比锁强在哪：锁只有开/关两态，只能管互斥；信号量带一个计数，能表达「还剩几个资源」，所以缓冲区有 N 个空位时能放行 N 个、第 N+1 个才排队。

最高频的辨析点是 S<0 时绝对值的含义：它是**等待进程数**，不是「已分配的资源数」——这是选择题常设的干扰项。务必同时记住这个结论的前提是**记录型信号量**（有等待队列、阻塞让权）；整型信号量靠忙等，值不变负，谈不上「负值含义」。

互斥用初值 1、同步用初值 0/n 是判初值的总抓手：互斥信号量的 P 和 V 一定在同一个进程里成对夹住临界区；同步信号量恰恰相反，V 在「先做的事」那个进程、P 在「后做的事」那个进程，靠这一前一后强制次序（「前 V 后 P」）。

> 给定一串 P/V 后求某时刻 S 的具体取值、前驱图的逐条边演算属于推导，交给题。卡里只锚「P/V 骨架、S 值三段含义 + 记录型限定、互斥/同步初值口诀」。

同步与互斥基本概念 高频 易错辨析

同步 vs 互斥（最爱考的辨析）：

关系	又称	含义
同步	**直接**制约	进程因协作需按特定**先后次序**执行
互斥	**间接**制约	多进程竞争同一资源，同一时刻只允许一个进入临界区

临界区访问**四原则**：空闲让进 / 忙则等待 / 有限等待 / 让权等待。
（前两条最基本；「让权等待」=不能进就放 CPU，是最易被违反的一条。）

临界区结构（顺序固定）：**进入区 → 临界区 → 退出区 → 剩余区**。

实现方法两类：

- 软件：单标志 / 双标志（先检查、后检查）/ Peterson
- 硬件：中断屏蔽 / TestAndSet(TSL) / Swap(XCHG)
- **除中断屏蔽外，所有软硬件方法都不满足「让权等待」→ 忙等（busy waiting）**。Peterson 满足前三原则、唯独不满足让权等待。

同步和互斥的区别是选择题的常驻陷阱：**互斥是「竞争」**（谁用都行、只是不能同时用），**同步是「协作」**（A 必须在 B 之前）。常见错误是把生产者-消费者只当互斥——它其实**同时含**互斥（缓冲区互斥访问）和同步（先生产后消费）。

四原则怎么记不混：判断一个互斥算法对不对，先卡「空闲让进 + 忙则等待」这两条基本盘，再看「有限等待」（会不会饿死）和「让权等待」（等待时是不是在空转 CPU）。

软硬件方法的「忙等」是高频考点：单标志法违反空闲让进（必须交替进入）；双标志先检查违反忙则等待（可能两个一起进）；双标志后检查会死锁；Peterson 修好了前三条但 while 循环仍是忙等。中断屏蔽法是唯一不忙等的，但只适用单处理机、且关中断是内核态特权指令。

> 各方法逐行代码的执行序列推演、违反哪一条的具体交错过程，属于推导，交给题与可视化。卡里只锚「同步/互斥定义、四原则、四区结构、方法清单 + 谁不满足让权等待」这层结论。

进程间通信 中频 对比表

四种进程间通信方式对比：

方式	速度	同步谁负责	方向
共享存储	**最快**	**进程自己**（需自加互斥）	双向
消息传递	中	OS 自动	双向
管道 pipe	中	**自动同步**	**半双工**
信号 signal	快	异步通知	单向

- **共享存储最快**（直接读写内存、内核不介入），但代价是**互斥要自己解决**（如用信号量）。
- **管道 = 半双工**（数据单向流），自带同步：满则写者阻塞、空则读者阻塞；FIFO 字节流，读过即消失。
- 消息传递分**直接**（指明对方 PID）/ **间接**（信箱中转，多对多更灵活）。
- **信号只传一个编号、不带数据内容**，是异步事件通知，不归入「传数据」那一类。

四种方式的考点几乎全在「速度」和「同步谁来管」这两列的取舍上，先把这张表的对立关系记牢。

核心对立——快 vs 省心：

- **共享存储最快**，因为两进程直接读写同一块物理内存、OS 建好映射后就撒手不管。但「OS 撒手」的另一面就是**互斥要进程自己加**（两个进程同时写会乱），通常配信号量。「最快」和「要自己处理同步」是同一枚硬币的两面。
- **管道**反过来——慢一点但**自动同步**：管道满了写进程自动阻塞、空了读进程自动阻塞，不用你操心。记住它是**半双工**（水管只能单向流，要双向得开两根管），且数据 FIFO、读过就没了。

两个易错小点：

- 信号（signal）**只传一个信号编号、不携带数据**，是「通知事件发生」的异步机制，和共享存储/消息传递/管道这种「搬运数据」的不是一个层面——别把它当数据通道。
- 消息传递的「直接 vs 间接」：直接通信发送方要指明接收进程 PID；间接通信走**信箱**中转，多个收发方可共用一个信箱，更灵活。

  PV 操作属于低级通信、信号的具体记录/发送/处理流程等细节见博客。

进程状态与转换 中频 易错辨析

五状态 + 关键转换（背「哪些转换合法/不可能」）：

五状态：创建 / 就绪 / 运行 / 阻塞 / 终止。

转换	主/被动	触发
运行→阻塞	**主动**	自己请求等资源/事件（I/O、P 操作）
阻塞→就绪	**被动**	等待的事件到来（I/O 完成、V 操作）
运行→就绪	**被动**	时间片到 / 被更高优先级抢占
就绪→运行	—	被调度程序选中

- **不可能的转换**（选择题陷阱）：阻塞 ×→ 运行（必先到就绪）；就绪 ×→ 阻塞（没在运行，发不出请求）；终止 ×→ 任何状态。
- **运行→就绪是被动**：除时间片到，**被更高优先级进程抢占**也算（别只记时间片一种）。
- 七状态：加**就绪挂起 / 阻塞挂起**（换出到外存）。挂起=换到外存，阻塞=在内存等事件，两者正交。

这张卡只考一件事：**判断一个状态转换合不合法**。把「主动 vs 被动」「合法 vs 不可能」两组辨析钉死，选择题就稳了。

两条单向转换记牢方向：

- **运行→阻塞是主动**（进程自己说「我要等 I/O」），**阻塞→就绪是被动**（事件来了，OS 把它捞回就绪队列）。注意阻塞醒来只能回**就绪**，不能直接回**运行**——因为 CPU 可能正被别人占着，必须重新排队等调度。
- 反过来「就绪→阻塞」不可能：进程在就绪态根本没在 CPU 上跑，发不出 I/O 请求这类阻塞动作。

最容易漏的整改点——运行→就绪有两种触发：

- 时间片用完（被动让出）；
- **被更高优先级进程抢占**（抢占式调度下也是被动）。只记「时间片到」会在「下列哪种情况发生运行→就绪」这类题上漏选。

挂起 vs 阻塞别混：阻塞是「我在等快递、干不了别的」（进程映像在**内存**），挂起是「我被换出到外存走廊待着了」。所以七状态的两个新态是**就绪挂起**和**阻塞挂起**——挂起这个维度和阻塞与否是正交的。

> 各状态的本质含义、队列组织等铺垫见博客；具体调度选哪个进程交给调度算法那张卡和题。

经典同步问题 低频 步骤口诀

三大经典问题，背「信号量设置骨架 + 为什么」，逐行代码默写交给大题。

① 生产者-消费者

- 信号量：mutex=1（缓冲区互斥）、empty=n（空位数）、full=0（满位数）。
- 骨架：生产者 `P(empty); P(mutex); 放; V(mutex); V(full);`，消费者 `P(full); P(mutex); 取; V(mutex); V(empty);`。
- ⚠️ **关键铁律：P 的嵌套顺序必须「先 P 资源信号量(empty/full)、再 P 互斥(mutex)」。反了会死锁**（缓冲区满时，生产者已持 mutex 却阻塞在 P(empty)，消费者拿不到 mutex → 互锁）。**V 的顺序任意，不影响正确性**。

② 读者-写者（读优先）

- 信号量：rw_mutex=1（读写互斥）、mutex=1（保护 readcount）、readcount=0。
- 骨架：**第一个**读者 `P(rw_mutex)` 挡住写者，**最后一个**读者 `V(rw_mutex)` 放行；readcount 的增减必须放在 mutex 保护内。
- 特点：读者-读者不互斥（可同时读），读-写、写-写互斥。读优先 → 写者可能**饥饿**。

③ 哲学家进餐

- 朴素方案：5 人同时拿左筷 → 都等右筷 → **死锁**。
- 防死锁三招：**限制人数**（最多 n-1 人同时拿）/ **奇偶拿筷**（奇号先左后右、偶号先右后左，破坏循环等待）/ **同时拿两只**（mutex 包住两次 P，破坏请求并保持）。

这三道题是 OS 大题主战场，但记忆卡只背**信号量怎么设、为什么这么设**，逐行代码靠刷题练手。

生产者-消费者的命门是 P 的嵌套顺序：实现互斥的 `P(mutex)` 一定要排在实现同步的 `P(empty)` / `P(full)` **之后**。一旦把 `P(mutex)` 提到前面，缓冲区满时生产者就会「持锁阻塞」——自己拿着 mutex 却卡在 P(empty) 上，消费者永远拿不到 mutex 去腾空间，整个系统死锁。记住这条的同时也记住对照面：**V 操作的顺序怎么排都不影响正确性**。

读者-写者的关键是 readcount 这个共享变量必须用 mutex 保护，而「第一个读者上锁、最后一个读者解锁」让多个读者能并发读。读优先方案的代价是写者可能被源源不断的读者饿死。

哲学家进餐本质是死锁演示：朴素写法每人先拿左筷就会循环等待。三招防死锁各破坏一个死锁必要条件——限人数和奇偶拿筷破坏「循环等待」，同时拿两只破坏「请求并保持」。

> 给定具体进程数/缓冲区大小写出完整 PV 代码、判断某个交错序列会不会卡死，属于推导，交给题与可视化。卡里只锚三套信号量骨架与「为什么」。

上下文切换 低频 定义锚点

上下文切换：

- **定义**：保存当前进程的 CPU 现场（PC / 寄存器 / PSW / 栈指针 → 存入 PCB）+ 恢复下一进程的现场。
- 是**纯开销**——切换过程 CPU 不做任何有用计算。
- **进程切换 ⊇ 上下文切换**，且代价大：要切**地址空间**（换页表基址寄存器、刷 TLB、Cache 失效）。
- **模式切换（即"态切换"，用户态↔内核态）≠ 进程切换**：模式切换只改 PSW 模式位、不换进程、更轻。
- 系统调用只引起**模式切换**，未必引起进程切换（没触发调度则当前进程不变）。

上下文切换的考点集中在「**和模式切换的区别**」「**它是纯开销**」两条，把这两条锚死即可。

先记定义：上下文 = 进程跑到一半时 CPU 里的各寄存器值（PC 执行到哪、寄存器算到一半的中间值、PSW 状态、栈指针），就像同时炒两道菜，切灶台时要记住每道菜炒到哪一步。切换 = 把当前进程这些现场**存进它的 PCB**，再从下一进程的 PCB **恢复**出来。

最高频辨析——上下文切换 vs 模式切换：

- **模式切换**：用户态↔内核态，只改 PSW 里的模式位，**进程不变**，开销小。
- **进程切换**：换的是进程，**必发生在内核态**，还要额外切**地址空间**——换页表基址寄存器、刷 TLB、Cache 大概率全失效，所以开销远大于模式切换。
- 由此推出一道经典陷阱题的答案：「系统调用会导致上下文切换吗」——**不一定**。系统调用只是用户态→内核态→用户态的**模式切换**，如果没顺带引发进程调度，当前进程根本没换，谈不上进程的上下文切换。

为什么是纯开销：切换那段时间 CPU 全在搬寄存器、刷 TLB，不算任何业务结果。这正是**时间片不能设太短**的原因——切得太勤，开销占比就高了。

> 切换的完整步骤流程、开销来源细节见博客与可视化。

管程与条件变量 低频 定义锚点

管程（Monitor） = 封装「共享数据 + 一组操作过程 + 同步」的高级机制。
组成：共享数据结构 + 一组过程 + 初始化语句 + 管程名。

关键特性：

- **互斥自动**：同一时刻最多一个进程在管程内执行，由**编译器保证**（程序员不用手写 P/V 管互斥）。
- **封装**：共享数据只能经管程的过程访问。

条件变量 cond（配 wait/signal 做等待/唤醒）：

- `wait(c)`：当前进程在 c 上阻塞，并**释放管程使用权**。
- `signal(c)`：唤醒在 c 上等待的一个进程。

条件变量 ≠ 信号量（高频辨析）：cond **没有计数值**，`signal` 时若无进程等待则**信号丢失**（空操作、不累积）；而信号量 V 无论有无等待者都使 value+1（信号被记录）。

管程是为了解决「信号量 P/V 太容易写错」而提出的更安全的同步抽象：把共享数据和操作打包，互斥交给编译器自动保证，程序员只需关心同步逻辑（什么时候该等、该唤醒）。

最该钉死的辨析是条件变量和信号量的区别：条件变量是「纯通知」，没有值—— `signal` 发出时如果没人在等，这个信号就**白白丢失**；而信号量的 V 操作会把计数加一、把信号「存」下来，后来者执行 P 时还能感知到。这是选择题常设的对照点。

另一个易混点是 wait 的语义：执行 `x.wait()` 时进程**主动阻塞自己并交出管程使用权**，好让别的进程能进管程——而不是占着管程空等。

> 管程内部各过程的具体同步实现、Hoare 与 Mesa 唤醒语义（signal 后谁先跑、要不要用 while 重检）属于进阶推导，交给博客精讲与题。卡里只锚「管程定义/三特性、条件变量 wait/signal、cond 与信号量的差别」。

线程实现 冷门 对比表

用户级线程 ULT vs 内核级线程 KLT：

维度	用户级线程 ULT	内核级线程 KLT
谁管理	用户空间线程库	操作系统内核
内核可见	不可见（只看到一个进程）	可见
切换	不进内核态，**快**	需进内核态，**慢**
一个线程阻塞	**整个进程阻塞**	只阻塞该线程，他人照跑
多核并行	**不能**（内核只调度进程）	**能**

- 只有 **KLT 才能被 OS 调度上 CPU**；ULT 必须映射到 KLT 才获得 CPU 时间。
- 多线程模型：**多对一**（多 ULT→1 KLT）/ **一对一**（1→1）/ **多对多**（m≥n）。

ULT 和 KLT 的所有差异都源于一个根问题：**内核到底知不知道这些线程的存在**。把这个根抓住，对比表里每一行都能自己推出来。

用一句话锚定：ULT「内核不可见」就像公司内部的小组分工——HR（内核）系统里只有一个「部门」（进程），部门内谁干啥 HR 不知道也不管。由此推出：

- 切换快（不用惊动内核，库里改个 PC 和栈就行）；
- 但**一个线程阻塞 → 整个进程阻塞**（内核只看到进程，进程一发阻塞 syscall 就把整个进程挂起，库没法替其他线程求情）；
- 也**无法多核并行**（内核只把「进程」这一个调度实体派给一个核）。

KLT 反过来：内核挨个认识每个线程，于是单线程阻塞不连累他人、能分到多核，代价是切换要陷入内核、开销大。

多线程模型记法：多对一=纯 ULT（继承 ULT 缺点：一阻全阻、不能多核）；一对一=纯 KLT（并发好但创建开销大）；多对多=折中（m 个 ULT 复用 n 个 KLT，m≥n）。

> 此 KP 暂无配套可视化。线程库的具体调度机制等细节见博客。

内存管理 9 张

页面置换算法 高频 对比表

四种页面置换算法（**只背规则与特性，不背引用串算缺页次数**）：

算法	淘汰规则	性能	Belady 异常	实现/特点
OPT 最佳	淘汰**未来**最久不用的	理论下界（最优）	无	**不可实现**（需预知未来），仅作性能基准
FIFO	淘汰**最早进入**的	最差	**可能有**	最简单（队列）；可能淘汰常用页
LRU	淘汰**过去**最久未使用的	接近 OPT	无	用过去预测未来；需硬件/栈支持，**开销大**
CLOCK / 改进 CLOCK	用**访问位/修改位**循环扫描近似 LRU	接近 LRU	**可能有**	低开销折中，实际系统常用

- **Belady 异常**（增加页框数缺页反而增多）的判据：**栈算法（LRU、OPT）一定无 Belady；非栈算法（FIFO、CLOCK 系列）才可能出现**。CLOCK 本质是 FIFO 变体、不是栈算法，所以同样可能有。注意「不会增加」≠「一定减少」（可能不变）。
- LRU = 看**过去**，OPT = 看**未来**，两者互为镜像。
- 改进 CLOCK 用 (访问位 A, 修改位 M) 分类，**优先淘汰未修改页**（脏页换出要写回，I/O 贵）。

这张卡只背「四算法的规则 + 特性对比」。给定引用串和页框数、模拟 FIFO/LRU/OPT/CLOCK 算缺页次数和命中过程——那是计算，**一律交给题和可视化，本卡绝不背**。

Belady 异常的归因务必说准：判据是**「是否栈算法」**，不能笼统说「FIFO 类才有」。**LRU、OPT 是栈算法，理论上保证增加页框数缺页次数不会增加（故无 Belady）**；**FIFO、CLOCK 系列不是栈算法，可能出现 Belady 异常**（增加页框数缺页反而变多）。还有个细抠点：栈算法增加页框「不会增加」缺页，但也不保证「一定减少」，可能持平——选项写「一定减少」是错的。

LRU 和 OPT 是一对镜像：OPT 淘汰「未来最久才用到」的页，因为要预知未来访问序列所以**不可实现**，只当性能天花板；LRU 把它翻过来，用「过去最久没用」近似预测未来，是可实现算法里最接近 OPT 的，但要给每页维护时间戳/栈、**硬件开销大**，所以实际系统退而求其次用 CLOCK。

CLOCK 为什么是折中：LRU 准但贵、FIFO 便宜但差，CLOCK 用一个**访问位 A** 做循环扫描（A=0 淘汰，A=1 清零后给「第二次机会」继续扫），低成本逼近 LRU。**改进 CLOCK** 再加**修改位 M**，按 (A,M) 分四类，优先淘汰「未访问且未修改」的页——因为脏页（已修改）换出时要写回磁盘，I/O 代价高，先扔干净页能省一次写盘。

> 任何引用串的缺页次数模拟、命中过程、改进 CLOCK 的逐轮扫描全部交给题和配套可视化，本卡只锚规则与定性特性。

页式管理 高频 定义锚点

分页：进程逻辑空间和物理内存都切成等大块（页/页框），靠页表记映射（**背机制结构，不背逻辑→物理地址数值换算**）。

- **逻辑地址 =（页号 P，页内偏移 W）**：页大小是 **2 的幂**，故偏移位数固定、地址按高低位天然拆分。
- **页表**存「页号 → 物理块（页框）号」映射；**页表寄存器**存页表起始地址 + 页表长度。
- **访存次数**：

情况	访存次数
基本分页（无快表）	**2 次**（查页表 + 取数据）
TLB（快表）命中	**1 次**（只取数据，查 TLB 不算访存）
TLB 未命中	**2 次**（查页表 + 取数据）
n 级页表 + TLB 未命中	**n+1 次**

- 页大小 2 的幂 → **有内碎片**（最后一页填不满）、**无外碎片**。
- 分页对用户**透明**、是**一维**地址（用户感知不到分页）。

这张卡背「分页的机制与结构」：地址怎么拆、页表存什么、访存几次、碎片哪类。**给定页大小和逻辑地址、手算页号/偏移/物理地址那套数值换算不背**，交给题和可视化。

为什么页大小一定是 2 的幂：这样页内偏移恰好占地址的低 k 位（页大小 = 2^k），页号占高位，CPU 直接按位拆分逻辑地址，不用做除法取模。也正因等大切分，永远不会有「块与块之间的缝」→ 无外碎片；但进程最后一页很可能装不满 → 有内碎片。固定等大 = 内碎片，这点和分段（外碎片）正好相反。

访存次数是最高频陷阱，钉死三句：基本分页要 **2 次**（先查内存里的页表拿页框号，再访问数据）；引入 **TLB（快表）** 后命中只要 **1 次**——TLB 是 Cache 级硬件，查它不计入访存；TLB 未命中退回 2 次。多级页表把这个数推广成 **n 级 + 未命中 = n+1 次**。常见错误是把「查 TLB」也算一次访存，记住 TLB 命中只访问数据那一次。

一维 vs 透明：分页是系统为提高内存利用率自动切的，用户写程序时根本不知道有页，地址在用户看来是连续的一维空间——这与分段（用户可见、二维）的根本区别要分清。

> 逻辑地址→物理地址的位拆分与 EAT 数值计算交给题和可视化；多级页表的逐级查表过程点「展开博客精讲」。

连续分配 中频 对比表

连续分配 = 给进程分一整块连续内存。三类方式 + 动态分区四算法（**背特点，不背具体分配过程**）：

方式	碎片
单一连续（系统区+用户区，单道）	有内碎片、无外碎片
固定分区（预划若干固定分区）	**有内碎片**、无外碎片
动态分区（按需分配）	无内碎片、**有外碎片**（靠紧凑/Compaction 合并）

动态分区四算法：

算法	规则	空闲表排列	特点
首次适应 First Fit	从头找第一个够大的	地址递增	**综合性能最好、开销小、最常用**；低地址积小碎片
最佳适应 Best Fit	选最小的够用块	大小递增	留**极小外碎片最多**（名不副实）
最坏适应 Worst Fit	选最大的块	大小递减	破坏大空间，大进程后期分不到
邻近适应 Next Fit	从上次结束位置接着找	—	分布较均匀，但也会拆大块

这张卡只背「方式 × 碎片」和「四算法的定性特点」。给定空闲分区表 + 进程序列、问各算法分到哪块、剩多少——那是计算过程，**交给题和可视化，本卡不背**。

碎片对应关系是核心考点：固定分区按固定大小切，进程比分区小 → 分区内部用不满 → **内碎片**；动态分区按进程实际大小切，反复分配回收后块间留下零碎空隙 → **外碎片**，用**紧凑（Compaction）**把进程挪到一起腾出大块连续空间。选择题最爱把内/外碎片对调，记死「固定→内、动态→外」。

四算法怎么记不混：按「空闲表怎么排」反推规则——首次适应按**地址**递增排（从低地址往后找第一个够的）；最佳适应按**大小**递增排（队头就是最小够用块）；最坏适应按**大小递减**排（队头是最大块）。一个反直觉结论要钉死：**「最佳适应」反而最差**——它每次抠出刚好够用的块，剩下的边角料极小、几乎无法再用，留下最多不可用外碎片；实践中**首次适应综合性能最好**。

> 各算法的逐步分配演练（命中哪块、剩余多少）用配套可视化，数值过程交给题。

内存管理基础 中频 定义锚点

内存管理三件事：**地址变换（重定位）+ 内存保护 + 共享/分配**。

- **逻辑（虚拟）地址 vs 物理地址**：程序里用逻辑地址（从 0 编址），运行时经地址变换映射到物理地址。
- **三种装入/重定位**：

方式	转换时机	可否移动
绝对装入	编译时定地址	不可（仅单道）
静态重定位	装入时一次改完	**不可移动**
动态重定位	运行时靠重定位（基址）寄存器	**可移动**（须硬件支持）

- **内存保护**：① 上、下限寄存器（检查 下限 ≤ 物理地址 ≤ 上限）；② 基址寄存器 + 限长寄存器（**先比逻辑地址 < 限长，再加基址**，越界则地址越界异常）。
- **碎片**：内碎片 = 分到手但没用上（分配单位偏大，如分页）；外碎片 = 空闲但太小用不上（动态分区）。
- 内存共享 = 多进程共享同一物理页/段（如只读代码段、动态库 .dll/.so）。

这张卡管「概念定义」——逻辑/物理地址、三种装入、两种保护、两类碎片。**地址换算的位拆分、具体加减数值不背**，那是分页卡和题目的活，这里只钉定性结论。

动态重定位为什么必须硬件支持：运行时每访存一次都要把逻辑地址翻译成物理地址，只有 CPU 自带的重定位寄存器 + 加法器能在一个访存周期内完成，软件做不到逐次实时翻译。也正因为它在运行时才转换，进程可以在内存里被搬动（紧凑），这是静态重定位做不到的——静态重定位装入时一次性算死物理地址，搬走就全错。

两种保护怎么记不混：上下限寄存器存的是「物理地址的上界和下界」，直接夹住合法区间；基址+限长寄存器存的是「起始物理地址 + 长度」，检查顺序是**先限长后基址**（先确认逻辑地址 < 限长，再加基址得物理地址）——这个先后是选择题陷阱，反过来写就是错的。

内碎片 / 外碎片别认反：固定划分（分页页框等大、固定分区）→ 末尾填不满 → **内**碎片；可变划分（动态分区、分段）→ 块与块之间留缝 → **外**碎片。一句话：固定→内，可变→外。

> 逻辑地址→物理地址的具体位拆分与数值换算交给分页卡和题目，本卡只锚机制与定义。重定位/保护的细节流程点「展开博客精讲」。

页框分配与回收 中频 易错辨析

页框分配 = 决定「每个进程分多少页框、缺页时从哪换」（**背概念与组合，不背最少页框数/最大文件等数值计算**）。

- **工作集**：某段时间窗口内进程实际访问的页面集合；**驻留集**：实际分给进程的页框数。**驻留集 ≥ 工作集**，否则频繁缺页（**抖动/颠簸**）。
- **分配策略两维交叉**（固定/可变分配 × 全局/局部置换）：

组合	是否存在
固定分配 + 局部置换	✓
可变分配 + 全局置换	✓（最易实现、最灵活，很多 OS 用）
可变分配 + 局部置换	✓（按缺页率动态调，效果最好）
固定分配 + 全局置换	**✕ 不存在**（全局置换会改变各进程页框数，与"固定"矛盾）

- **最少页框数**由**指令集架构**决定（一条指令 + 跨页操作数/间接寻址需几页）。
- **调入时机**：预调页（按空间局部性预取相邻页，首次装入用）vs 请求调页（缺页才调，运行中为主）。

这张卡背「工作集/驻留集概念 + 四种分配组合 + 调入策略」。**最少页框数、最大文件、抖动临界点这些具体数值不背**，交给题。

最高频陷阱 = 排除「固定分配 + 全局置换」：全局置换允许从别的进程抢页框，这会改变各进程实际拥有的页框数，和「固定分配（页框数运行期不变）」直接矛盾，所以这个组合**不存在**。剩下三种都可行，记法：固定只能配局部（自己换自己）；可变可配全局或局部。

工作集 vs 驻留集别混：工作集是「这段时间真正访问到的页的集合」（客观需求，由程序行为决定）；驻留集是「OS 实际分给它的页框数」（供给）。供给 < 需求（驻留集 < 工作集）就会反复缺页换页 → **抖动（颠簸）**。所以分配原则是驻留集别小于工作集。

最少页框数看指令集：不是随便定的下限——一条指令本身占 1 页，它的操作数若跨页、或用间接寻址要多访问一页，就得同时驻留更多页框，否则一条指令都执行不完。具体几页是题目算的，本卡只背「由指令集/寻址方式决定」。

> 最少页框数、抖动临界、缺页率调节的数值计算交给题；页面缓冲、页框回收机制点「展开博客精讲」。

虚拟内存（OS 视角） 中频 定义锚点

虚拟内存：基于**局部性原理**，程序只装入一部分即可运行，逻辑容量「看起来」很大。

- **理论基础 = 局部性原理**：时间局部性（刚访问的很快再访问）+ 空间局部性（访问位置附近的很快被访问）。
- **三特征**：**多次性**（分多次调入）/ **对换性**（暂不用的换出外存）/ **虚拟性**（逻辑上扩充内存，最本质）。多次性、对换性是手段，虚拟性是目的。
- **实现必须基于离散分配**（请求分页/请求分段/请求段页式）；连续分配**无法**实现虚拟内存。
- 容量上限 = **min(内存+外存总容量, 地址位数寻址空间)**，不是「等于外存大小」。
- **请求分页**：访问的页不在内存（状态位 P=0）→ 触发**缺页中断**，调入后**重新执行**该指令。
- 缺页中断属 **故障类异常（Fault，内中断）**，**在指令执行期间产生**（非执行完后）；一条指令操作数跨页可触发多次缺页。**缺页 ≠ 越界中断**（越界是访问非法地址，缺页是合法但不在内存）。

这张卡背「虚拟内存的概念定义、三特征、缺页中断的性质」。EAT/缺页率的数值计算**交给题**，本卡只锚定性结论。

三特征别背串：**多次性**（不必一次全装入，可分多次调）、**对换性**（运行中可换进换出）是两个手段，**虚拟性**（从逻辑上把内存撑大、用户感觉内存远比物理大）才是目的。考点常问「最本质的特征」——答虚拟性。

缺页中断的性质是 power user 试金石，钉死三点：① 它是**内中断里的故障类（Fault）**，不是外部设备引起的外中断；② 它**在一条指令执行的中途**产生（普通中断在指令执行完才查），所以处理完要**重新执行**这条指令；③ **缺页 ≠ 越界**——越界中断是访问了不属于自己的非法地址（该报错），缺页是访问的是合法页、只是暂时不在内存（调进来即可继续）。把这两个混为一谈是经典错选项。

为什么必须离散分配：虚拟内存要按页（段）独立换进换出，连续分配整块装入根本拆不开，所以「连续分配也能实现虚拟存储」是错的。另外容量上限取「内外存总量」和「地址位数」两者的**较小值**，不要写成等于外存大小。

> 缺页率、有效访存时间 EAT 的数值演算交给题；缺页中断完整处理流程点「展开博客精讲」。

内存映射文件 低频 定义锚点

内存映射文件（mmap）：把文件**映射进进程虚拟地址空间**，之后像访问内存一样用**指针**读写文件，不再调 read/write。

- **机制基于请求分页**：调 `mmap()` 时**不立即读入**，访问到才触发缺页中断按需调页；脏页在换出/退出时写回文件。
- **优点**：① 减少用户态/内核态间的数据拷贝（不经内核缓冲区→用户缓冲区两次拷贝）；② 多进程映射**同一文件**可实现**共享内存**（比消息传递高效）。
- 适用大文件、随机访问；常规 read/write 适合顺序/小量读写。
- 共享映射：改动对其他进程可见并写回文件；私有映射：写时复制（Copy-on-Write），改动不影响他人。

这张卡背「mmap 的定义 + 与常规 I/O 的区别 + 它和虚拟内存的关系」。低频考点，通常作为选择题某个选项出现。

和常规 read/write 的根本区别：常规文件 I/O 每次都走「系统调用 → 内核缓冲区 → 拷到用户缓冲区」，数据要拷两道；mmap 把文件直接接到虚拟地址空间，进程用指针访问，省掉用户态/内核态之间的数据拷贝。这是它「快」的来源。

它本质是请求分页的一个应用，别当成独立机制：mmap 调用时只建立映射关系、**不真读数据**，等进程访问到映射区才缺页调入；内存不足时脏页按虚存的页面置换换出（写回文件）。所以它和虚拟内存共用同一套缺页/置换机制。

为什么能做进程通信：多个进程把同一个文件映射进各自地址空间，就共享了同一份物理页框 → 一种共享内存实现，比走消息传递的拷贝高效。共享映射改动互相可见，私有映射走写时复制各改各的。

> mmap 的系统调用细节与页表建立过程点「展开博客精讲」，本卡只锚概念与对比。

段式管理 低频 对比表

分段：按程序逻辑模块（代码段/数据段/栈段）划分，段长不等。

- **逻辑地址 =（段号 S，段内偏移 W）**，是**二维**地址（用户可见）。
- **段表**存「段号 →（段长，段基址）」；访问要**两次越界检查**：① 段号 S < 段表长度 ② 段内偏移 W < 段长。
- 段长不等 → **有外碎片、无内碎片**。

分页 vs 分段（选择题必考对比）：

维度	分页	分段
划分依据	系统按**固定大小**自动切（物理单位）	用户按**逻辑模块**切（逻辑单位）
大小	固定	可变
地址空间	**一维**	**二维**
碎片	内碎片	外碎片
共享/保护	不方便（函数可能跨页）	**方便**（一段=一个逻辑单元）
对用户	透明	可见

这张卡核心是「分段定义 + 分页/分段对比表」。给定段表手算物理地址、判越界——那是计算，**交给题**，本卡只锚机制和定性区别。

为什么分段方便共享和保护：段是按逻辑功能切的（一个完整函数就是一个代码段），要共享只需共享对应的段、要保护按段设权限即可；分页把同一个函数切碎散在多个页里，共享和保护都对不齐边界。这是「分段=逻辑单位、便于共享保护」的根因。

别和分页认反的三组对比：维度（页=物理单位、段=逻辑单位）、地址（页一维、段二维）、碎片（页内碎片、段外碎片）。记忆抓手——分页是系统替你切、你看不见（一维透明、固定大小→内碎片）；分段是你按模块自己切、段长不等（二维可见、可变大小→外碎片）。

两次越界检查是分段的标志：分页只查一次（页号 < 页表长度），分段要查两次（段号越界 + 段内偏移超过该段段长）——因为段长不固定，必须用段表里记的段长再卡一道。

> 段表查表算物理地址、越界判定的数值过程交给题和可视化。

段页式管理 冷门 定义锚点

段页式 = **先分段（逻辑）→ 每段再分页（物理）**，兼有分段便于共享保护 + 分页消除外碎片的优点。

- **逻辑地址 =（段号 S，段内页号 P，页内偏移 W）**，三部分；对用户是**二维**（用户只见段号和段内偏移，页号对用户透明）。
- **段表**不再存段基址，改存「**页表起始地址 + 页表长度**」；每段一张页表。
- **访存次数（无 TLB）= 3 次**：查段表 → 查页表 → 取数据。故段页式**必配快表**。
- 碎片：**有内碎片**（段内最后一页填不满）、**无外碎片**。

这张卡背「段页式的地址结构、表结构、访存次数、碎片」。给定段表/页表手算物理地址那套数值换算**交给题**，本卡只锚机制。

怎么理解「先分段再分页」：把书先按章节拆开（分段，保留逻辑边界便于共享保护），再把每一章裁成统一大小的卡片（分页，消除外碎片）。查找时先定位是第几章（查段表），再定位章内第几张卡片（查页表），最后翻到具体一行（页内偏移）——这就是三次访存的来由。

两个易混点钉死：① 段页式的**段表存的是页表地址**，不是段基址（因为段内不再连续存放，而是分了页，要靠页表二次映射）；② 碎片是**内碎片**——段内已经分页了，外碎片被消掉，但每段最后一页仍可能填不满。和纯分段（外碎片）正好相反。

访存 3 次 → 必配快表：无 TLB 时一次数据访问要先后碰段表、页表、数据三块内存，开销是基本分页（2 次）的 1.5 倍，所以段页式系统几乎都靠 TLB 把段号+页号一次命中、把访存压回接近 1 次。

段表/页表查表算物理地址的数值过程交给题和可视化。

文件管理 7 张

文件基本概念 高频 定义锚点

- 文件 = 一组有意义信息的集合，有名字、是逻辑外存的最小分配单元。
- 元数据（文件属性）存于 **FCB / inode（索引节点）**：含文件大小、权限、时间戳、物理块指针等。
- **inode 不含文件名**——文件名只在「目录项」里（目录项 = 文件名 + inode 号）。这是硬链接能共享同一 inode 的前提。
- 文件操作：创建 / 删除 / **打开（建立"文件↔进程"连接，把目录项信息放入打开文件表）** / 关闭 / 读 / 写 / 定位。
- 读写指针放在**进程打开文件表**里（不在 inode、也不在系统打开文件表）——不同进程各自独立。
- 「系统打开文件表里同一文件只有一项」说的是**属性只有一份**（再有进程打开只让打开计数 +1），**不是读写指针只有一份**。一句话锚：**属性一份，指针各一份**。

---

inode 里到底有没有文件名，是这章选择题最爱设的坑。把元数据（大小 / 权限 / 块指针）和文件名拆开放，是为了让目录检索更快：目录项只存「文件名 + inode 号」十几个字节，一个磁盘块能塞下更多目录项；同时也让「一个 inode 对应多个文件名」（硬链接）成为可能。**记住一句：文件名在目录项里，其余属性在 inode 里。**

怎么记不混：

- **打开文件表分两级**——系统打开文件表（全系统共享，记 FCB 信息 + 打开计数）、进程打开文件表（每进程私有，记 fd → 系统表索引 + 读写指针）。「文件读写指针存哪」答**进程打开文件表**，因为同一文件被多进程打开时各自的位置互不相干。
- **open 在做什么**：搜目录找到目录项 → 信息复制进打开文件表 → 返回文件描述符 fd，之后读写凭 fd 直接定位，不必每次再搜目录。
- **为什么会读到"每次 open 都新增一个表项"的相反说法**：真实实现是**三层**——fd 表 →「打开文件描述」（每次 open 新建一项，读写指针在这层）→ 内存索引结点（每个文件唯一一份，带打开计数）。考试用的两级模型把中间层并进了用户打开文件表，于是「系统打开文件表」指最底层、按文件唯一；而孙钟秀《操作系统教程》把这个名字给了中间层（按 open 新建），汤小丹《计算机操作系统》干脆只讲一张不分级的打开文件表。**同名不同层，两种说法各有教材依据**——所以别背名字，背上面那句「属性一份、指针各一份」：os-2020-23 考前半句，os-2017-31 考后半句。
- 「分配空间」按**簇 / 块**为最小单位，再小的文件也至少占一簇——文件大小 ≠ 实际占用磁盘空间（向上取整到整簇，余下是内部碎片）。

  簇数 = ⌈文件大小 / 簇大小⌉ × 簇大小 这类**数值换算交给题**，卡里只锚「按整簇分配、有内部碎片」这个定性结论。

目录管理 中频 易错辨析

- 目录项 = 文件名 → FCB/inode 的映射；多级（树形）目录便于分类、允许不同目录下重名。
- **硬链接 vs 软链接（符号链接）对比**：

特性	硬链接	软链接（符号链接）
本质	多个目录项指向**同一 inode**	独立文件，内容是**目标路径字符串**
inode	共享同一个	有自己独立的 inode
引用计数	创建时 inode 计数 +1	不影响目标的计数
跨文件系统	**不可以**（inode 号仅本 FS 内有效）	**可以**
链接目录	**不可以**（防遍历成环）	**可以**
删除目标后	其他硬链接仍可访问	链接**悬空**，访问报错

- 删硬链接 = 删目录项 + inode 计数 −1，**计数减到 0 才真正回收** inode 与数据块。

硬链接和软链接的区别是这个 KP 的必考点，关键全在「删除行为」和「能不能跨界」两组对比上。

怎么记不混：

- **硬链接 = 给同一个 inode 多取一个名字**：多个目录项共享同一 inode，谁都不是「正本」。删掉其中一个名字只是计数 −1，只有计数归 0、且没有进程还开着它，才真正释放数据。
- **软链接 = 快捷方式**：它是个独立小文件，里面存的是目标的路径字符串。原文件被删，软链接就成了「悬空链接」，照路径找不到东西。
- **为什么硬链接不能跨文件系统、不能链目录**：inode 号只在本文件系统内唯一，跨 FS 就指不准；链接目录会让目录树可能形成环，遍历时死循环。软链接存的是路径不是 inode 号，所以两条限制都不受。
- 删除文件时「不可能执行的操作」类反向题：删文件 = 删它的目录项 / FCB + 回收空间，但**不会去删它所在的那个目录**。

  路径解析需要多少次磁盘 I/O（约 2n 次）这类**数值计算交给题**，卡里只锚结构与删除行为。

文件物理结构 中频 对比表

三种分配方式对比（综合题核心，**背优缺点不背块号计算**）：

分配方式	顺序存取	随机存取	外部碎片	文件增长	适用
连续分配	最快（磁头不动）	支持（起始块+i）	**有**	困难（要预知大小）	文件大小固定
链接·隐式链接	支持	**不支持**（须沿链逐块）	无	方便	顺序存取文件
链接·显式链接(FAT)	支持	**支持**（FAT 常驻内存可遍历）	无	方便	FAT 文件系统
索引分配	支持	支持	无	方便	Unix/Linux

- **隐式链接 ✕ 随机 / 显式链接(FAT) ✓ 随机**——选择题高频陷阱，钉死。
- 索引分配：索引块存该文件所有块号；缺点是**小文件也占一整个索引块（浪费）**、**大文件需多级 / 混合索引**（Unix inode = 12 直接 + 一次/二次/三次间接）。
- 连续分配唯一有**外部碎片**的；链接、索引均无外部碎片。

物理结构是这章出计算大题最密集的地方，但记忆卡只锚「每种方式支不支持随机存取、有没有外部碎片、能不能动态增长」这套定性对比，把数值算法留给题。

怎么记不混：

- **连续分配**像在书架上预留一整排连续格子：读最快（磁头不移动）、能直接算第 i 块位置，但中间插不进新书（不易扩展），且会在磁盘上留下大小不一的空洞（外部碎片）。
- **隐式链接**像寻宝游戏，每块只告诉你下一块在哪——想读第 5 块必须从头跑，所以**不支持随机存取**；指针还散落在各数据块里，一处损坏后面全断。
- **显式链接**把所有指针抽出来集中放进一张 **FAT 表**并常驻内存，于是在内存里就能快速遍历整条链，**支持随机存取**——这是它与隐式链接最关键的差别。
- **索引分配**为每个文件单独建索引块存全部块号，天然支持随机存取、无外部碎片；代价是小文件也得占一个索引块，大文件一个索引块装不下要靠多级/混合索引。

  混合索引算最大文件大小、给定偏移量算磁盘 I/O 次数、块号占字节换算——这些**数值计算交给题与可视化**，卡里不背中间值。

文件系统全局结构 低频 定义锚点

- 一个磁盘分区内的文件系统布局（自前向后）：
  引导块 → 超级块 → 空闲空间管理区 → inode 区 → 数据块区。
- **引导块**：启动该分区上 OS 的引导代码。
- **超级块**：文件系统全局元信息——文件系统类型/魔数、块大小、总块数、**空闲块数**、总 inode 数、空闲 inode 数、根目录 inode 号等。
- **inode 区**：所有 inode 按编号排列；**数据块区**：目录与文件的实际数据。
- 磁盘最前端是 **MBR（主引导记录）= 引导程序 + 分区表**（分区表记各分区起止）。
- 常见文件系统：FAT（文件分配表式）、NTFS、ext。

这个 KP 把前面学的 inode、目录、空闲管理拼成一张完整的磁盘布局图，考点集中在「各区域分别存什么」和「超级块的作用」。

怎么记不混：

- **MBR 是整盘级、引导块是分区级**：MBR 在磁盘最前端，含分区表 + 总引导程序，决定从哪个分区启动；每个分区内部还各有自己的引导块，负责启动该分区上的 OS。别把两者混成一个。
- **超级块 = 文件系统的「出版信息页」**：记的是全局统计（总块数、空闲块数、inode 总数等），不是某个文件的信息。超级块一旦损坏整个文件系统就用不了，所以常存多个副本；它在挂载时读入内存、卸载时写回。
- **布局顺序**记成一条流水线：先「引导块」能启动 → 再「超级块」描述全局 → 「空闲管理区」记哪些块空着 → 「inode 区」记每个文件的元数据 → 最后「数据块区」放真实内容。

  FAT 表长度、最大文件大小这类**数值计算交给题**，卡里只锚布局与超级块内容。

外存空闲空间管理 低频 对比表

四种外存空闲空间管理方法对比：

方法	数据结构	空间开销	分配速度	适用
空闲表法	表：每个连续空闲区记「起始块号 + 块数」	中等	较快	连续分配
空闲链表法	空闲块（盘块链）或空闲区（盘区链）串成链	小	盘块链慢 / 盘区链中	离散分配
位示图法	每位对应一个块，0/1 表空闲/占用	小（N 块 = N bit）	快	各种分配方式
成组链接法	空闲表 + 链表结合，分组、组间链接	中等	快	Unix / 大型文件系统

- **位示图**：每位对应一块，便于扫连续 0 找连续空闲块、适合位运算；盘块号 ↔ 行号/列号可换算（**换算交给题**）。
- **成组链接法**（Unix）：第一组信息存在**超级块**里，每组最后一块存下一组的空闲块号，串成链；分配从栈弹出、回收压入栈。

这四种方法回答同一个问题——「文件删了之后，怎么记住哪些磁盘块空着」，考点是它们各自的结构特点和适用场景。

怎么记不混：

- **空闲表 / 空闲盘区链**都以「连续空闲区」为单位（起始块号 + 块数），适合连续分配；**空闲盘块链**以单个块为单位，分配多个块要逐个取，最慢。
- **位示图**是最经典的标志性结构：一位一块、0/1 表状态，占空间极小（N 块只要 N bit），找连续空闲块只需扫连续的 0，现代文件系统普遍在用。
- **成组链接法**像把空教室清单分页：第一页贴在教务处（超级块），每组的最后一块记着下一组在哪，于是既能快速分配（从当前组的栈里取）又能管海量空闲块。
- 易混点：**inode 是管「已分配文件的块」，不是管「空闲块」**——别把 inode 误当成空闲块管理结构。

  位示图盘块号 ↔ 字号/位号（行列）的具体换算、位示图所需字节数等**数值计算交给题**，卡里只锚结构对比与定性结论。

文件逻辑结构 冷门 对比表

逻辑结构 = 用户视角看文件怎么组织（与磁盘上怎么放的物理结构相互独立）。

逻辑结构	组织方式	顺序存取	随机（直接）存取	典型
无结构（流式文件）	字节流，无内部结构	支持	按字节偏移	.txt、可执行文件、Unix 几乎一切
顺序文件	记录按顺序排（串结构=按存入序 / 顺序结构=按关键字序）	最优	仅顺序结构可折半	批量处理、磁带
索引文件	每条记录建索引项，索引表按关键字排	支持	支持	变长记录、随机查询
索引顺序文件	分组 + 稀疏索引（每组一项）	支持	支持	大量记录的折中

- 串结构（按存入先后排）vs 顺序结构（按关键字排，可折半查找）——这是顺序文件内部的辨析点。
- 索引文件适合**变长记录**（定长记录直接算偏移即可，无须索引）。

逻辑结构回答的是「用户怎么看这堆数据」，和物理结构（数据在磁盘块上怎么摆）是两条独立的线——同一种逻辑结构可以落在不同物理结构上，别混。

怎么记不混：

- **无结构 = 字节流**，没有「记录」这个概念，只能按字节偏移定位；有结构 = 记录式，以「记录」为存取基本单位。
- **顺序文件**的两种排法决定了能不能折半：串结构按存入先后排，找记录只能逐个比（O(n)）；顺序结构按关键字排好序，能折半（O(log n)）。
- **索引顺序文件**是顺序文件和索引文件的折中——先在稀疏索引里定位到「哪一组」，再在组内顺序找。它解决的是「记录太多、纯顺序太慢、纯索引表又太大」的两难。

  各结构的平均检索长度（如索引顺序文件的 $\sqrt{N}$）属于计算推导，**交给题**，卡里只锚各结构「支不支持随机存取、适用什么场景」。

虚拟文件系统与挂载 冷门 定义锚点

- **VFS（虚拟文件系统）= 内核中的抽象层**：向上对用户提供统一的文件操作接口（open/read/write/close），向下屏蔽不同文件系统（ext / FAT / NTFS / NFS）的实现差异。
- 每个具体文件系统只需实现 VFS 定义的标准接口（如各自的 read 函数），即可接入系统——用户无须为不同 FS 用不同系统调用。
- VFS 的「虚拟」= 抽象层 / 统一接口，**与虚拟内存无关**；VFS 本身不存数据、不加快访问，只做请求转发。
- **挂载（mount）= 把一个文件系统接到目录树的某个挂载点上**，之后通过该路径访问；挂载点须是已存在的（通常为空）目录。

VFS 和挂载在 408 里属低频，了解概念即可，但「VFS 的虚拟到底指什么」这一句辨析容易被「虚拟内存」带偏。

怎么记不混：

- **VFS 像万能充电头**：对上统一是一个接口（系统调用 open/read/write），对下通过不同「转接头」适配各种文件系统（ext4、FAT32、NFS）。它的价值是「统一接口」，不是「加速」，也跟虚拟内存毫无关系——别被「虚拟」二字误导。
- **VFS 能访问网络文件系统**（如 NFS），不是只能访问本地——这也是常见的错误选项。
- **挂载是「接入目录树」**：U 盘插上后文件不会自动出现，要把它的文件系统挂到某个挂载点（如 /mnt）才能按路径访问；解析路径走到挂载点时，系统查挂载表自动「穿越」到目标文件系统，对用户透明。

  VFS 的四个核心对象（超级块/inode/dentry/file）、挂载表字段等细节属扩展了解，卡里只锚「统一抽象接口」与「挂载 = 接入目录树」两个定义锚点。

输入输出管理 7 张

I/O 控制方式 高频 对比表

① 四种 I/O 控制方式对比（演进方向＝CPU 干预越来越少）：

方式	传输单位	CPU 干预	数据流向	特点
程序查询（轮询）	字	全程**忙等**	设备→CPU→内存	最差，CPU 利用率最低
中断驱动	字	**每字节/字中断一次**	设备→CPU→内存	数据仍经 CPU，中断频繁
DMA	**块**	只在块传完中断一次	设备→**内存（不经 CPU）**	一次传一个连续块
通道	**一组块**	CPU 干预最少	设备→**内存**	独立处理机执行通道程序

② 中断处理次序（次序固定，勿乱）：

- **中断隐指令（硬件自动完成，非真指令）**：关中断 → 保存断点(PC/PSW) → 送中断向量（引出中断服务程序）。
- **中断服务程序（软件）**：保存现场和屏蔽字 → 开中断 → 执行服务 → 关中断 → 恢复现场和屏蔽字 → 中断返回。

⚠ 两个陷阱：**保存断点（硬件隐指令完成）≠ 保存现场（软件完成）；中断隐指令由硬件自动完成，不是一条真正的机器指令。**

四种控制方式背的是"CPU 被解放了多少"这条主线：程序查询全程忙等最差 → 中断驱动每个字节/字中断一次（数据还得经 CPU 中转）→ DMA 以块为单位、数据不经 CPU 直接进内存、只在整块传完才中断一次 → 通道是独立的 I/O 处理机、执行通道程序、一次能传一组块、CPU 干预最少。

关键区分：程序查询和中断驱动，数据都要"设备→CPU 寄存器→内存"绕一圈；DMA 和通道，数据**直接**在设备和内存之间走，不经 CPU。中断驱动每个**字**中断，DMA 每个**块**中断——这一字一块是选择题的死穴。

中断处理次序的两个陷阱（power user 试金石，务必钉死）：

- **第一段是硬件自动完成的"中断隐指令"**：关中断 → 保存断点 → 送中断向量。「关中断」必须排最前，它给"保存断点"这段上锁，否则保存到一半来了新中断会把刚存的断点覆盖。中断隐指令不是程序员能写的真指令，是硬件在中断响应周期里自动做的一串动作。
- **保存"断点"≠保存"现场"**：保存断点（PC/PSW）由硬件隐指令完成；保存现场（通用寄存器、屏蔽字）由中断服务程序这段软件完成，发生在已经关中断之后。混为一谈是经典错点。

  ⚠ 减法：背四方式对比 + 中断骨架就够，**磁盘寻道距离、缓冲区处理时间这类数值计算不进卡，交给题**。DMA 控制器寄存器组成（MAR/DC/DR）等细节点「展开博客精讲」。

I/O 软件层次结构 高频 对比表

I/O 软件四层结构（自下而上）：

层（自下而上）	与硬件	干什么
① 中断处理程序	与硬件中断机制相关	响应 I/O 完成中断，保存/恢复现场，唤醒驱动
② 设备驱动程序	**与硬件相关**（唯一）	把抽象 I/O 请求翻译成具体设备命令，操作控制器寄存器
③ 设备独立性（无关）软件	与硬件无关	**命名、保护、缓冲、分配** + 统一接口、差错处理
④ 用户层 I/O	用户态	库函数（printf/scanf）、SPOOLing

- **设备独立性 = 用户用「逻辑设备名」而非物理设备名**，便于更换设备、上层不关心是磁盘还是 SSD。
- **驱动程序是唯一直接和设备硬件细节打交道的层**（运行在内核态）。

四层从下往上背：中断处理程序 → 设备驱动程序 → 设备独立性（设备无关）软件 → 用户层 I/O。记住两个"唯一"就不会错：**驱动程序是唯一与具体硬件相关的层**（每种设备一套驱动），**设备独立性软件是把"和设备无关的活"统一收口的层**。

怎么记不混：

- 设备独立性软件那四件事——**命名、保护、缓冲、分配**——是高频选择项，记成"命保缓分"。它对上提供统一接口、屏蔽设备差异，让上层读文件时不必知道存在磁盘还是 SSD。
- 「设备独立性 / 设备无关性」的落点就是**逻辑设备名 → 物理设备名的映射**：用户程序只写逻辑名，换设备时只改映射、不动应用代码。
- 用户层那层别漏 **SPOOLing**（假脱机部分实现在用户层）和库函数。

  一次完整 I/O 请求从 read() 到磁头动起来的逐步流程属理解性内容，点「展开博客精讲」看；卡里只锚四层名称 + 各层职责骨架。

磁盘调度 中频 对比表

磁盘访问时间 = **寻道时间 + 旋转延迟 + 传输时间**（寻道占主导，调度目标＝减少寻道）。

五种磁盘调度算法对比（背规则与特点）：

算法	规则	饥饿	特点
FCFS	按请求到达顺序	无	公平，寻道可能很差
SSTF	每次选离磁头**最近**的请求	**可能**（远端磁道挨饿）	寻道较优但不公平
SCAN（电梯）	沿一个方向走，**到边界才折返**	无	来回扫，刚扫过的方向要等一个来回
C-SCAN	单向服务，**到头直接回起点**重来（返回不服务）	无	响应更均匀、最公平
LOOK / C-LOOK	同 SCAN/C-SCAN，但**到最远的请求处就折返**（不到物理边界）	无	实际系统常用

磁盘调度背的是"五种算法各自的规则 + 会不会饥饿 + 公不公平"，外加访问时间三段式构成。瓶颈在寻道，所以调度目标是让磁头少跑冤枉路。

怎么记不混：

- **SSTF 是唯一会饥饿的**——它老挑最近的，远端磁道可能长期没人理；FCFS/SCAN/C-SCAN/LOOK 都不饥饿。
- **SCAN vs LOOK 是计算题最大陷阱**：SCAN（电梯）必须走到**物理边界**（如 0 或最大磁道号）才折返；LOOK 只走到**最远的那个请求**就折返。题面写"SCAN"就到边界，写"LOOK"就到最远请求，两者总寻道距离不同。
- **C-SCAN vs SCAN**：C-SCAN 是单向服务，到头后直接跳回起点重新单向扫（返程不服务），让各请求等待时间更均匀、最公平；SCAN 是来回双向都服务。
- 访问时间三段：**寻道 + 旋转延迟 + 传输**，记住寻道占大头即可。

> ⚠️ 减法：**给定磁道请求序列算总寻道距离/平均寻道、旋转延迟具体数值——全是计算，不进卡，交给题和可视化**。卡里只锚算法规则与特点。

缓冲区管理 低频 对比表

引入缓冲的目的：① 缓和 CPU 与 I/O 速度不匹配 ② 减少中断次数 ③ 解决数据粒度不匹配 ④ 提高 CPU 与 I/O 并行性。

四种缓冲对比：

缓冲	缓冲区数	特点
单缓冲	1	处理与传输**串行**（取数据 M 期间设备不能写）
双缓冲	2	设备写一块、CPU 读另一块，可**并行**；支持**双向同时通信**
循环缓冲	多块环形	设备/CPU 各一指针在环上追赶，平滑速度差
缓冲池	系统统一管理、双向共享	三队列：**空缓冲队列 / 输入队列 / 输出队列**

- 磁盘高速缓存 vs 缓冲区：前者存近期访问的磁盘数据副本、加速**重复访问**（可长期驻留）；后者暂存正在传输的数据、协调速度差（取走即释放）。

缓冲区是 OS 给"CPU 几纳秒处理一个字、磁盘几毫秒读一块"这道速度鸿沟搭的桥。背的核心是**单缓冲、双缓冲、循环缓冲、缓冲池**四级递进，以及缓冲池那三个队列。

怎么记不混：

- 单缓冲只有一块，传入和取走共用它，所以"取数据"期间设备得停着等——处理和传输是串行的；双缓冲两块错开用，设备和 CPU 真正并行，还能双向同时收发。这是单/双缓冲的本质差别。
- 缓冲池是"公共池子按需分配"，记牢三队列：**空缓冲队列 / 输入队列 / 输出队列**，配"收容输入、提取输入、收容输出、提取输出"四种操作。
- 别把**磁盘高速缓存**和**缓冲区**搞混：缓存是为了重复读不再访盘（数据留着），缓冲是为了协调一次传输的速度差（数据取走就还回去）。

  ⚠️ 减法：**单/双缓冲每块处理时间的 max(T,C)+M / max(T,C+M) 数值计算不进卡，交给题**。卡里只锚"哪种能并行、缓冲池几个队列"这类定性结论。

I/O 设备分类与接口 冷门 对比表

I/O 设备分类（按信息交换单位是考点）：

维度	分类	特点
信息交换单位	**块设备**（磁盘、SSD）	以块为单位、**可寻址、可随机访问**、传输率高
信息交换单位	**字符设备**（键盘、打印机、串口）	以字符为单位、**不可寻址、顺序访问**
共享属性	独占（打印机）/ 共享（磁盘）/ 虚拟（SPOOLing 改造）	独占同一时段只给一个进程

- I/O 端口编址两种：**独立编址**（专用 I/O 指令 in/out，端口有独立地址空间）vs **统一编址/内存映射**（端口映射进内存地址空间，用普通访存指令）。
- 设备控制器 = CPU 与设备的中介，含数据寄存器 / 状态寄存器 / 控制寄存器 + I/O 逻辑。

设备分类里**最常被拿来出陷阱的是「块设备 vs 字符设备」**：判定标准是"信息交换的单位"，不是"设备快慢"。块设备以块为单位、可按块号寻址，所以能随机访问（磁盘、SSD）；字符设备以单个字符为单位、不可寻址、只能顺序收发（键盘、打印机、串口）。

怎么记不混：

- 「可不可寻址」是块/字符设备的本质区别——块设备每块有块号能直接定位，字符设备是字符流没有"地址"概念。
- I/O 端口两种编址，记**靠不靠用专用指令**：独立编址要专用 I/O 指令、不占内存地址；统一编址（内存映射）直接用访存指令读写设备寄存器、要占一段内存地址空间。
- 「独占 → 共享」的桥梁是 SPOOLing（虚拟设备），这条线索把本卡和假脱机那张卡串起来。

  设备控制器各寄存器的具体读写时序、阻塞/非阻塞 I/O 的实现细节属理解性内容，点「展开博客精讲」看，卡里只锚分类骨架。

假脱机 SPOOLing 冷门 定义锚点

- **SPOOLing（假脱机）= 用软件 + 磁盘上的输入井/输出井，把独占设备（如打印机）虚拟成多台共享设备。**
- 组成：**输入井 / 输出井**（磁盘上的缓冲区）+ **输入缓冲区 / 输出缓冲区**（内存）+ **输入进程 / 输出进程。**
- 效果：多个用户「同时」用打印机——进程把打印数据写进输出井就立即返回，输出进程在后台按队列顺序真正送去打印（实际请求排队）。
- 本质：用**软件 + 磁盘**模拟"脱机"，与主机**同时**运行（脱机是专用外围机 + 磁带、在主机运行前后）。

SPOOLing（假脱机）要锚死的是它的定义和"独占 → 共享"那条主线：打印机本是独占设备一次只能服务一个进程，SPOOLing 在磁盘上开"输出井"当排队缓冲，进程把数据丢进井里就以为打完了、立即继续执行，后台的输出进程再按顺序把井里的任务一个个送进打印机。于是多个用户感觉自己在"同时"用打印机。

怎么记不混：

- 组成三对，记成"两井、两缓冲、两进程"：**输入井/输出井在磁盘上**（容量大、当排队区），**输入/输出缓冲区在内存里**（中转），**输入/输出进程**负责搬运。井在磁盘、缓冲在内存——别记反。
- SPOOLing 的价值是把**独占设备虚拟成共享设备**（虚拟设备），还顺带消除了"因抢打印机导致死锁"的隐患。
- 和"脱机技术"对照记：脱机靠专用外围机 + 磁带、在主机运行前/后干活；SPOOLing 靠软件 + 磁盘、与主机**同时**运行，是用软件模拟脱机的效果。

  SPOOLing 在 I/O 软件层次里部分实现于用户层，与「设备独立性软件」「假脱机如何避免死锁」的展开见博客精讲，卡里只锚定义 + 组成。

SSD 读写特性与磨损均衡 冷门 定义锚点

- SSD 基于**闪存**，无机械部件、随机访问快、**不需要磁盘调度**（无寻道时间）。
- 读写以**页**为单位、擦除以**块**为单位（一块含多页）；**"先擦后写"——写前必须整块擦除，不能直接覆盖写。**
- 每个块**擦除次数有限**（寿命），故需**磨损均衡（Wear Leveling）**：让各块擦写次数尽量平均，避免某块被写穿。分**动态均衡**（新数据写到擦得少的块）与**静态均衡**（把冷数据搬走、腾出少擦块给热数据）。
- **FTL（闪存翻译层）** 做逻辑地址 ↔ 物理地址映射。

SSD 这张卡锚四件事：读写擦的单位、不能覆盖写、磨损均衡、FTL。它没有机械臂，随机访问比机械盘快上千倍，也因此**不存在寻道时间、用不上 SCAN/SSTF 这类磁盘调度算法**（内部调度交给 FTL）。

怎么记不混：

- **"读写按页、擦除按块"** 是最常考的一句：读写的最小单位是页（一块含若干页），擦除只能一整块来。
- **"不能覆盖写、先擦后写"** 是 SSD 的特殊性根源——修改一个页往往要连累整块擦了重写，这也是写入放大与寿命问题的来由。
- 块的擦除次数有限，所以要**磨损均衡**：动态均衡把新数据写到擦得少的块，静态均衡把冷数据搬走、腾出擦得少的块给热数据。目的是别让某几块先被写穿。
- **FTL** 负责逻辑→物理地址映射，磨损均衡和坏块管理都靠它在底层调度。

  SLC/MLC/TLC/QLC 寿命对比、写入放大的具体机理属理解性延伸，点「展开博客精讲」看；卡里只锚"页读写、块擦除、先擦后写、磨损均衡、FTL"这几个硬结论。

扫码回站内背这套卡

站内按记忆曲线排期，到期自动进「今日待背」